



EXPERIMENT 2

Student Name: Ridhima Gulati

Branch: AIT-CSE (Cyber Security)

Semester: 5

Subject Name: ADBMS

UID: 24BCY70132

Section/Group: 24AIT_KRG-G2

Date: 13-07-2026

Subject Code: 24CSP-333

EXPERIMENT: Advanced SQL Queries

AIM: To create and populate relational database tables and perform SQL queries using joins, aggregate functions, GROUP BY, ORDER BY, UNION ALL, and correlated subqueries to analyze data.

Question1:(A)

A university administration requires an analytical performance report for each academic unit. Write a PostgreSQL solution to calculate the average marks scored by students grouped by their respective departments. The solution must join the student profile, course subjects, and grade marks tables, returning a breakdown sorted by the highest average marks down to the lowest.

Table:

```
CREATE TABLE Departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Students (  
    student_id INT PRIMARY KEY,  
    student_name VARCHAR(50) NOT NULL,  
    dept_id INT REFERENCES Departments(dept_id)  
);
```

```
CREATE TABLE Subjects (  
    subject_id INT PRIMARY KEY,  
    subject_name VARCHAR(50) NOT NULL
```

EXPERIMENT 2

);

```
CREATE TABLE Marks (  
    student_id INT REFERENCES Students(student_id),  
    subject_id INT REFERENCES Subjects(subject_id),  
    marks_scored INT NOT NULL,  
    PRIMARY KEY (student_id, subject_id)
```

);

```
INSERT INTO Departments (dept_id, dept_name) VALUES  
(1, 'Computer Science'),  
(2, 'Mathematics');
```

```
INSERT INTO Students (student_id, student_name, dept_id) VALUES  
(101, 'Alex', 1),  
(102, 'Bella', 1),  
(103, 'Charlie', 2);
```

```
INSERT INTO Subjects (subject_id, subject_name) VALUES  
(501, 'Data Structures'),  
(502, 'Calculus');
```

```
INSERT INTO Marks (student_id, subject_id, marks_scored) VALUES  
(101, 501, 85),  
(101, 502, 90),  
(102, 501, 75),  
(103, 502, 95);
```

Answer:

```
Select d.dept_name as Department, ROUND(avg(m.marks_scored),2) as avg_marks  
from Departments d  
join Students s  
on d.dept_id=s.dept_id  
join marks m  
on s.student_id=m.student_id
```

EXPERIMENT 2

group by d.dept_id, d.dept_name
order by avg_marks desc;

	department character varying (50) 	avg_marks numeric 
1	Mathematics	95.00
2	Computer Science	83.33

Question 2: A multinational organization maintains separate records for employee information and department details. The management wants to identify the employees who receive the highest salary within their respective departments for an annual performance report.

If more than one employee earns the same highest salary in a department, all such employees should be included in the report. The final output should display the Department Name, Employee Name, and Salary, arranged in ascending order of department name.

Note: The employee and department information are maintained in separate normalized tables.

Table: CREATE TABLE Department

```
(
  id INT PRIMARY KEY,
  dept_name VARCHAR(50)
);
```

CREATE TABLE Employee

```
(
  id INT PRIMARY
  KEY, name
  VARCHAR(50),
  salary INT,
  department_id INT,
  FOREIGN KEY(department_id)
  REFERENCES Department(id)
);
```

EXPERIMENT 2

```
INSERT INTO Department
VALUES
(1,'IT'),
(2,'SALES');
```

```
INSERT INTO Employee
VALUES
(1,'JOE',70000,1),
(2,'JIM',90000,1),
(3,'HENRY',80000,2),
(4,'SAM',60000,2),
(5,'MAX',90000,1);
```

Input Table: Employee

ID	NAME	SALARY	DEPT_ID
1	JOE	70000	1
2	JIM	90000	1
3	HENRY	80000	2
4	SAM	60000	2
4	MAX	90000	1

Department

ID	DEPT_NAME
1	IT
2	SALES

```
(i) SELECT
    d.dept_name,
    e.name,
    e.salary
FROM Employee e
JOIN Department d
```

EXPERIMENT 2

```

ON e.department_id = d.id
WHERE e.salary = (
  SELECT MAX(e1.salary)
  FROM Employee e1
  WHERE e1.department_id = e.department_id
)
ORDER BY d.dept_name;

```

Answer:

	dept_name character varying (50) 🔒	name character varying (50) 🔒	salary integer 🔒
1	IT	JIM	90000
2	IT	MAX	90000
3	SALES	HENRY	80000

(ii) Find the second highest earning employee from the same schema. SELECT

```

d.dept_name,
e.name,
e.salary
FROM Employee e
JOIN Department d
  ON e.department_id = d.id
WHERE e.salary = (
  SELECT MAX(e1.salary)
  FROM Employee e1
  WHERE e1.department_id = e.department_id
  AND e1.salary < (

```

EXPERIMENT 2

```

SELECT MAX(e2.salary)
FROM Employee e2
WHERE e2.department_id = e.department_id
)
)
ORDER BY d.dept_name;

```

Answer:

Showing rows: 1 to 2		Page No: 1	of 1
	dept_name character varying (50)	name character varying (50)	salary integer
1	IT	JOE	70000
2	SALES	SAM	60000

Question 3: Two legacy HR systems (A and B) have separate records of employee salaries. These records may overlap. Management wants to merge these datasets and identify each unique employee (by EmpID) along with their lowest recorded salary across both systems.

Objective: 1. Combine two tables A and B.

2. Return each EmpID with their lowest salary, and the corresponding Ename.

Table A

EmpID	Ename	Salary
1	AA	1000
2	BB	300

Table B

EmpID	Ename	Salary
2	BB	400
3	CC	100

CREATE TABLE

A (

EmpID INT,

Ename VARCHAR(50),

EXPERIMENT 2

```
Salary INT  
);
```

```
CREATE TABLE B  
(  
    EmpID INT,  
    Ename VARCHAR(50),  
    Salary INT  
);
```

```
INSERT INTO A (EmpID, Ename, Salary)  
VALUES  
(1, 'AA', 1000),  
(2, 'BB', 300);
```

```
INSERT INTO B (EmpID, Ename, Salary)  
VALUES  
(2, 'BB', 400),  
(3, 'CC', 100);
```

```
Answer: Select EmpID, Ename, min(Salary) as lowest_salary  
from  
(select EmpID, Ename, Salary  
from A  
union all  
select EmpID, Ename, Salary  
from B)  
as complete_table  
group by EmpID,Ename  
order by EmpID;
```

EXPERIMENT 2

	empid integer	ename character varying (50)	lowest_salary integer
1	1	AA	1000
2	2	BB	300
3	3	CC	100

Leetcode Question: 175

Table: Person

Column Name	Type
personId	int
lastName	varchar
firstName	varchar

personId is the primary key (column with unique values) for this table.

This table contains information about the ID of some persons and their first and last names.

Table: Address

Column Name	Type
addressId	int
personId	int
city	varchar
state	varchar

addressId is the primary key (column with unique values) for this table.

Each row of this table contains information about the city and state of one person with ID = PersonId.

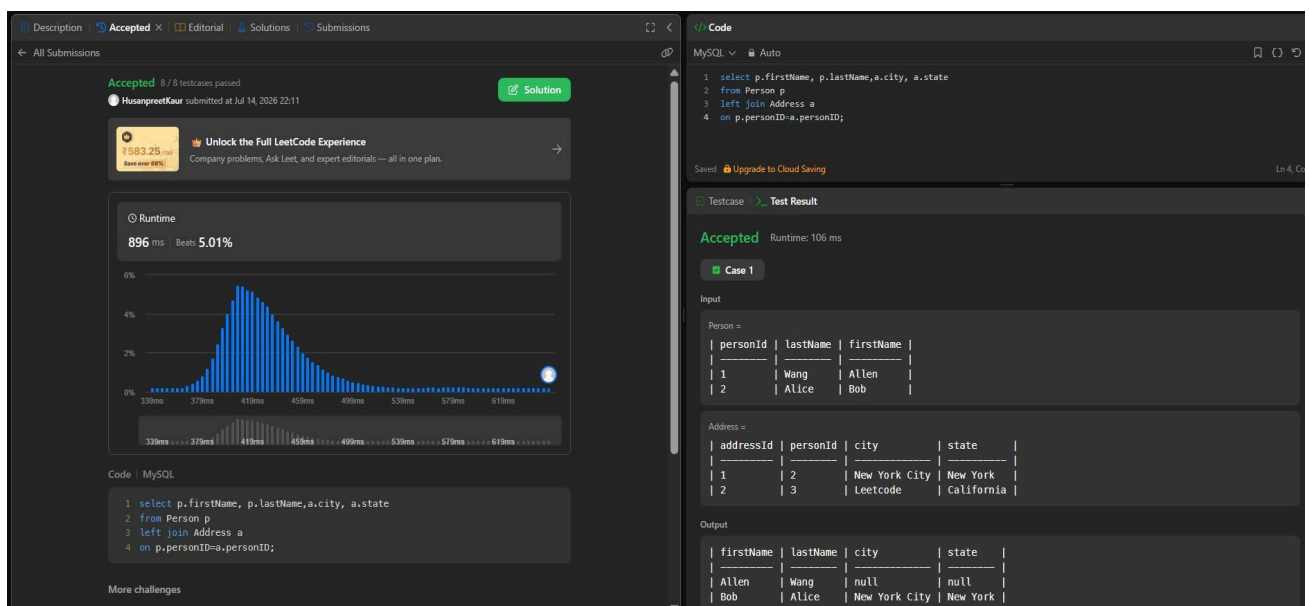
EXPERIMENT 2

Write a solution to report the first name, last name, city, and state of each person in the Person table. If the address of a personId is not present in the Address table, report null instead.

Return the result table in **any order**.

The result format is in the following example.

Answer: select p.firstName, p.lastName, a.city, a.state
from Person p
left join Address a
on p.personID=a.personID;



LEARNING OUTCOMES:

1. Learned how to create multiple related tables using SQL CREATE TABLE statements and apply constraints such as Primary Key, Foreign Key, and NOT NULL to maintain data integrity and establish relationships between tables.
2. Understood how to insert records into database tables and retrieve meaningful information by using different types of SQL joins and combining data from multiple related tables to generate accurate and useful reports.

EXPERIMENT 2

3. Gained hands-on experience with aggregate functions such as AVG(), MIN(), and MAX(), along with the GROUP BY and ORDER BY clauses, to summarize data, calculate statistics, and present results in a structured and meaningful manner.
4. Learned to combine data from multiple tables using the UNION ALL operator and write correlated subqueries to retrieve advanced results, such as the highest and second-highest salaries within each department.
5. Analyzed practical business scenarios, formulate appropriate SQL queries, and extract meaningful insights from relational databases. This enhances their ability to design efficient database solutions for academic, organizational, and industrial applications.